

ÉCOLE MAROCAINE DES SCIENCES DE
L'INGÉNIEUR (EMSI)

FILIÈRE : DDSIG1

RAPPORT DE PROJET ACADÉMIQUE DÉTAILLÉ

Plateforme de Gestion des Absences Multi-Rôles
Spécifications Fonctionnelles V2, Architecture JPA, Sessions & Cookies

Membres d'équipe :

Nizar Belkhadir
Adam Bibilou
Adam Asaas
Chafiq Mwiye

Présenté devant :

Pr. Houssam BAZZA

Module :

JEE et Génie Logiciel

Année Universitaire : 2025/2026

Table des matières

1	Introduction et Contexte du Projet	3
1.1	Problématique de l'assiduité universitaire	3
1.2	Choix Technologique : Java EE 8 pur et JPA	3
2	Cahier des Charges et Spécifications Fonctionnelles	3
2.1	Description des Rôles Applicatifs	3
2.1.1	Espace Administration (Admin)	3
2.1.2	Espace Enseignant (Professeur)	4
2.1.3	Espace Étudiant	4
2.2	Cycle de Vie des Demandes de Régularisation	4
3	Partie Conception (Modélisation UML)	6
3.1	Diagramme de Cas d'Utilisation (Use Case Diagram)	6
3.2	Diagramme de Classes (Class Diagram)	6
3.3	Diagramme de Séquence (Sequence Diagram)	6
4	Infrastructure, Persistance et Couche Modèle	8
4.1	Conteneurisation Database (PostgreSQL et Docker)	8
4.2	Configuration de JPA (<code>persistence.xml</code>)	8
4.3	Implémentation des Classes du Domaine (Modèles JPA)	9
5	Gestion du Cycle de Vie JPA et EntityManagers	10
5.1	Le Listener de Démarrage (<code>JPAInitializer.java</code>)	10
5.2	Thread Isolation des Managers (<code>JPAUtil.java</code>)	10
6	Couche de Persistance : Implémentation des DAOs	11
6.1	Gestion Manuelle des Transactions	11
6.2	Le Patron Data Access Object (DAO)	11
7	Couche Contrôleur : La Servlet Front-Controller	12
7.1	Vérification des Rôles et Session Gate	12
7.2	Structure Majeure du Servlet de Contrôle	12
8	Gestion d'État : Sessions & Cookies	13
8.1	Statelessness HTTP et HttpSession	13
8.2	Persistance et Remplissage via Cookies	13
9	Téléversement de Fichiers & Exportations	14
9.1	Téléversement avec Multipart Config	14
9.2	Exportation CSV avec Signature BOM	14
10	Interface Utilisateur (UI) & CSS Glassmorphism	14
10.1	Philosophie du Design System	14
10.2	Filtrage client temps réel (Vanilla JS)	15

11 Structure Détaillée des Vues (JSPs)	15
11.1 Espace Étudiant - student-dashboard.jsp	16
11.2 Espace Enseignant - prof-dashboard.jsp	16
11.3 Espace Administration - admin-requests.jsp	16
12 Déploiement et Configuration	17
12.1 Configuration de déploiement (web.xml)	17
12.2 Maven pom.xml	17
13 Scénarios d'Exécution et Recette de Tests	17
13.1 Test 1 : Authentification par rôle	17
13.2 Test 2 : Appel interactif professeur	17
13.3 Test 3 : Dépôt de justificatif et validation	17
14 Conclusion et Perspectives	18

1 Introduction et Contexte du Projet

1.1 Problématique de l'assiduité universitaire

Dans le cadre de la gestion académique au sein des établissements d'enseignement supérieur, le suivi de l'assiduité des étudiants représente un facteur déterminant pour garantir la rigueur pédagogique et lutter contre le décrochage scolaire. La saisie manuelle des absences, l'archivage physique des justificatifs médicaux et le calcul fastidieux des statistiques d'assiduité sont des tâches chronophages pour le corps administratif.

Un système défaillant ou obsolète engendre de nombreux problèmes :

- **Perte d'informations** : Perte de justificatifs papier déposés auprès de l'administration.
- **Erreurs de saisie** : Incohérences lors du report manuel des absences saisies par les enseignants.
- **Manque de réactivité** : Les étudiants ne sont informés de leurs absences qu'en fin de semestre, rendant impossible toute contestation rapide ou rattrapage.

Ce projet vise à relever ces défis en concevant une plateforme interactive multi-rôles permettant d'interconnecter directement l'administration scolaire, les professeurs et les étudiants, assurant un traitement transparent et collaboratif de l'assiduité.

1.2 Choix Technologique : Java EE 8 pur et JPA

Afin d'acquérir une compréhension profonde des mécanismes internes des architectures logicielles d'entreprise, nous avons opté pour le développement d'une application en **Java EE 8 pur (Jakarta EE 8)** sans recours aux frameworks d'auto-configuration de type Spring Boot. Cette approche permet de manipuler directement :

- Les cycles de vie des **Servlets** HTTP.
- Le rendu dynamique par **JSP** (JavaServer Pages) avec Expression Language (EL) et JSTL.
- La gestion de persistance par **JPA** (fournisseur Hibernate) avec requêtes JPQL.
- La conteneurisation de la base de données relationnelle via **PostgreSQL 15** sous Docker.

2 Cahier des Charges et Spécifications Fonctionnelles

2.1 Description des Rôles Applicatifs

L'architecture de l'application s'articule autour de trois rôles distincts, chacun disposant d'un espace applicatif sécurisé :

2.1.1 Espace Administration (Admin)

L'administrateur scolaire est le garant de la cohérence globale du système. Ses droits comprennent :

- Consultation du registre global de toutes les absences déclarées.
- Saisie manuelle d'une absence (par exemple, pour régulariser des séances particulières).
- Modération des demandes de justificatifs d'absence soumises par les étudiants.

- Modération des demandes de réclamation soumises par les étudiants.
- Exportation de l'historique complet des absences au format CSV pour Excel.

2.1.2 Espace Enseignant (Professeur)

L'enseignant est chargé de l'appel pour les cours qui lui sont attribués :

- Accès à la liste de ses cours et séances d'enseignement.
- Saisie rapide et interactive de la feuille d'appel pour une séance spécifique.
- Ajout automatique des absences pour les étudiants cochés absents, et suppression automatique si un étudiant est décoché.

2.1.3 Espace Étudiant

L'étudiant dispose d'un accès en lecture à ses propres données et peut formuler des demandes d'ajustement :

- Visualisation de ses statistiques personnelles (nombre total d'absences, absences justifiées, absences injustifiées).
- Dépôt d'une demande de justification (renseignant un motif textuel et une pièce justificative PDF/image optionnelle).
- Dépôt d'une réclamation pour signaler qu'il a été noté absent par erreur lors d'un cours où il était présent.

2.2 Cycle de Vie des Demandes de Régularisation

Pour assurer la modération sans supprimer les données arbitrairement, nous avons modélisé deux workflows distincts :

- **Flux de Justification** : L'étudiant soumet un motif et un fichier. L'absence passe au statut de justification **PENDING**. L'administrateur examine la demande : s'il l'accepte, le statut devient **ACCEPTED** et l'absence est considérée comme justifiée. S'il la refuse, le statut devient **REJECTED** et l'absence reste injustifiée.
- **Flux de Réclamation** : L'étudiant déclare sa présence indue. L'absence passe au statut de réclamation **PENDING**. L'administrateur valide : s'il accepte la réclamation, l'absence est supprimée de la base de données. S'il la rejette, le statut devient **REJECTED** et l'absence est maintenue.

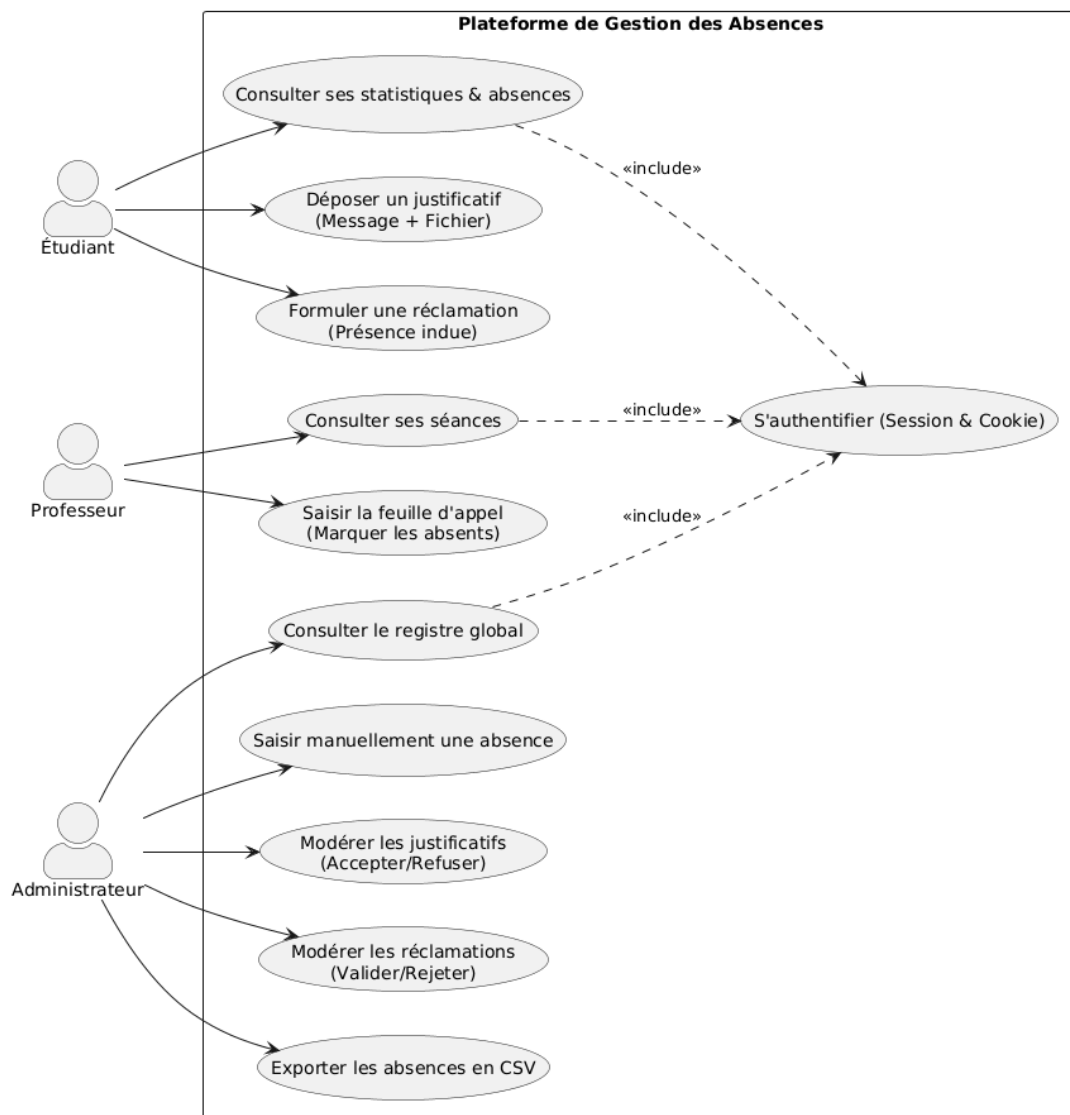


FIGURE 1 – Diagramme de Cas d'Utilisation - Plateforme de Gestion des Absences

3 Partie Conception (Modélisation UML)

3.1 Diagramme de Cas d'Utilisation (Use Case Diagram)

3.2 Diagramme de Classes (Class Diagram)

3.3 Diagramme de Séquence (Sequence Diagram)

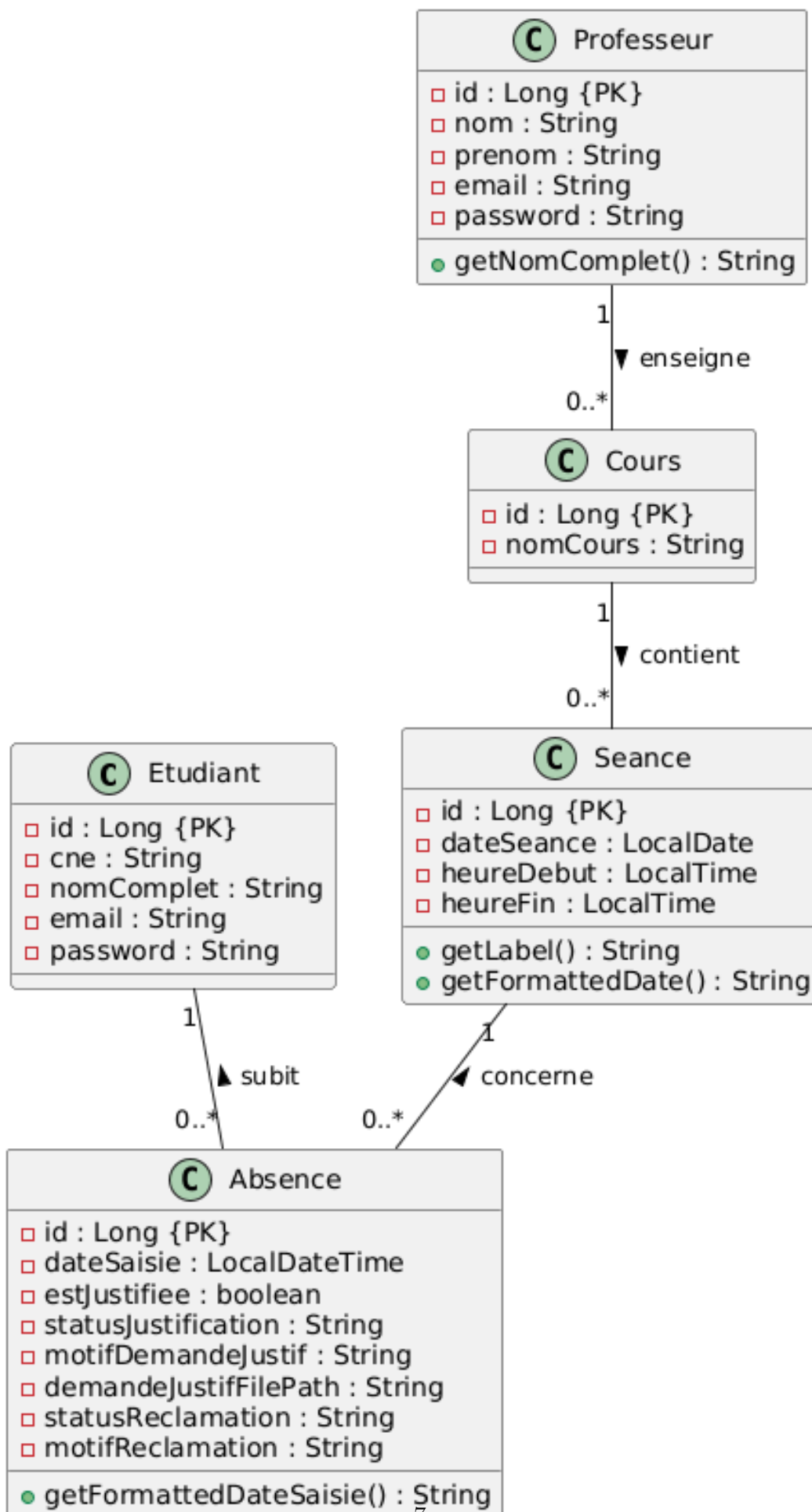


FIGURE 2 – Diagramme de Classes JPA - Modèle du Domaine

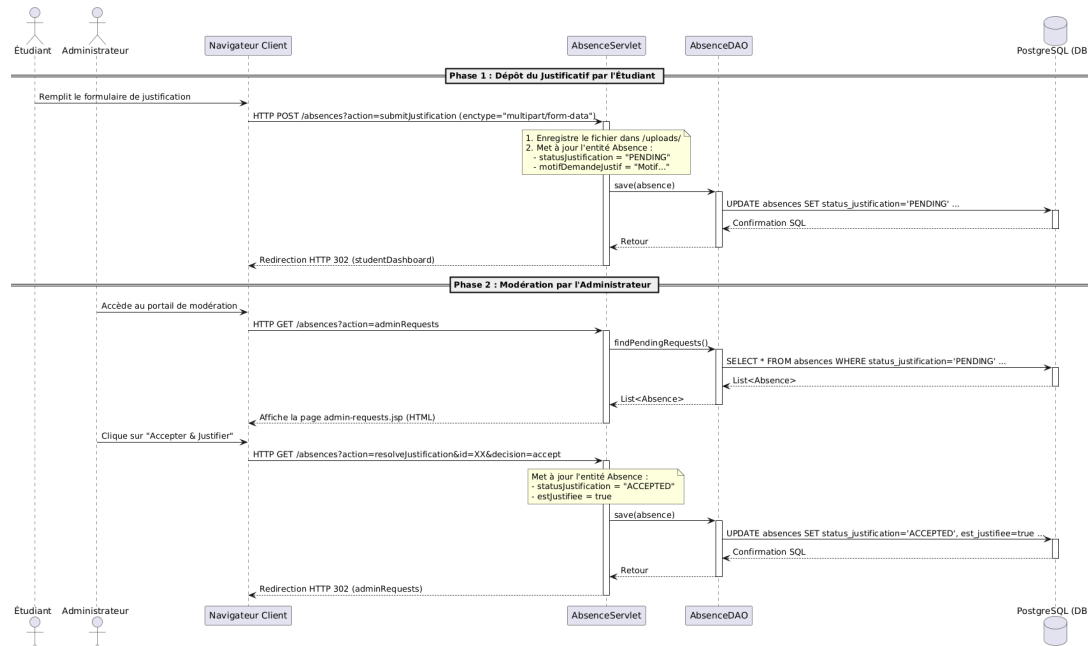


FIGURE 3 – Diagramme de Séquence - Traitement des Demandes et Persistance

4 Infrastructure, Persistance et Couche Modèle

4.1 Conteneurisation Database (PostgreSQL et Docker)

La base de données PostgreSQL est configurée via un fichier Docker Compose facilitant le déploiement sur le port physique local 5436.

```

1 version: '3.8'
2 services:
3   db:
4     image: postgres:15
5     container_name: postgres_jee_absences
6     environment:
7       POSTGRES_DB: gestion_absences
8       POSTGRES_USER: postgres
9       POSTGRES_PASSWORD: postgres
10    ports:
11      - "5436:5432"
12    volumes:
13      - pgdata:/var/lib/postgresql/data
14
15 volumes:
16   pgdata:
  
```

Listing 1 – Fichier docker-compose.yml

4.2 Configuration de JPA (persistence.xml)

L'unité de persistance JPA configure le driver PostgreSQL JDBC et la génération automatique du schéma de tables.

```
1 <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  version="2.1">
2   <persistence-unit name="AbsencePU" transaction-type="
  RESOURCE_LOCAL">
3     <provider>org.hibernate.jpa.HibernatePersistenceProvider<
  /provider>
4     <class>com.gestionabsences.model.Etudiant</class>
5     <class>com.gestionabsences.model.Cours</class>
6     <class>com.gestionabsences.model.Seance</class>
7     <class>com.gestionabsences.model.Absence</class>
8     <class>com.gestionabsences.model.Professeur</class>
9     <properties>
10      <property name="javax.persistence.jdbc.driver" value=
  "org.postgresql.Driver" />
11      <property name="javax.persistence.jdbc.url" value="
  jdbc:postgresql://localhost:5436/gestion_absences" />
12      <property name="javax.persistence.jdbc.user" value="
  postgres" />
13      <property name="javax.persistence.jdbc.password"
  value="postgres" />
14
15      <property name="hibernate.dialect" value="org.
  hibernate.dialect.PostgreSQL95Dialect" />
16      <property name="hibernate.hbm2ddl.auto" value="update
  " />
17      <property name="hibernate.show_sql" value="true" />
18      <property name="hibernate.format_sql" value="true" />
19    </properties>
20  </persistence-unit>
21 </persistence>
```

Listing 2 – Fichier persistence.xml

4.3 Implémentation des Classes du Domaine (Modèles JPA)

Dans le cadre de l'architecture JPA, chaque table de la base de données PostgreSQL est représentée par une classe Java annotée (Entity). Afin d'assurer une séparation propre des préoccupations (Clean Architecture), ces entités se concentrent sur le mapping relationnel.

Voici la description conceptuelle de nos cinq entités principales :

1. **Etudiant** : Modélise un étudiant identifié de manière unique par son CNE. Il possède des attributs de connexion (email et mot de passe) et son nom complet.
2. **Professeur** : Représente l'enseignant responsable des appels. Il possède un nom, prénom, email et mot de passe.
3. **Cours** : Représente une matière enseignée, associée à un professeur unique via une relation de type plusieurs-à-un (ManyToOne).
4. **Seance** : Représente un créneau horaire de cours (date, heure de début, heure de fin) rattaché à un cours.
5. **Absence** : C'est l'entité centrale du système. Elle lie un étudiant à une séance de cours spécifique et porte les statuts de justification et de réclamation.

Pour illustrer le paramétrage JPA de notre projet, nous présentons ci-dessous un extrait de la classe `Absence.java` montrant la configuration des clés étrangères, de la contrainte d'unicité (un étudiant ne peut avoir qu'une seule absence par séance) et des nouveaux champs de modération :

```
1 @Entity
2 @Table(name = "absences", uniqueConstraints = {
3     @UniqueConstraint(columnNames = {"etudiant_id", "seance_id"})
4 })
5 public class Absence implements Serializable {
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @ManyToOne(fetch = FetchType.EAGER)
11    @JoinColumn(name = "etudiant_id", nullable = false)
12    private Etudiant etudiant;
13
14    @ManyToOne(fetch = FetchType.EAGER)
15    @JoinColumn(name = "seance_id", nullable = false)
16    private Seance seance;
17
18    @Column(name = "status_justification", length = 20)
19    private String statusJustification = "NONE"; // NONE, PENDING
20    , ACCEPTED, REJECTED
21
22    @Column(name = "status_reclamation", length = 20)
23    private String statusReclamation = "NONE"; // NONE, PENDING,
24    REJECTED
25 }
```

Listing 3 – Extrait JPA de la classe `Absence.java`

5 Gestion du Cycle de Vie JPA et EntityManagers

5.1 Le Listener de Démarrage (`JPAInitializer.java`)

Tomcat étant un conteneur de servlets léger, il n'initialise pas nativement la persistance JPA au chargement. Nous y remédions en implémentant un `ServletContextListener` annoté `@WebListener`. Ce listener instancie l'unique `EntityManagerFactory` au démarrage du serveur et le ferme proprement à l'arrêt du contexte, évitant ainsi les fuites de ressources.

5.2 Thread Isolation des Managers (`JPAUtil.java`)

L'`EntityManager` n'étant pas thread-safe, nous utilisons la classe `ThreadLocal` de Java pour associer une instance d'`EntityManager` distincte à chaque thread de requête HTTP traitée par le pool de Tomcat.

La classe utilitaire `JPAUtil` centralise cette logique, permettant à toutes les classes DAO d'obtenir de façon sécurisée le gestionnaire de persistance lié au thread courant :

```
1 public class JPAUtil {
2     private static EntityManagerFactory emf;
3     private static final ThreadLocal<EntityManager>
        threadLocalEntityManager = new ThreadLocal<>();
4
5     public static EntityManager getEntityManager() {
6         EntityManager em = threadLocalEntityManager.get();
7         if (em == null || !em.isOpen()) {
8             em = emf.createEntityManager();
9             threadLocalEntityManager.set(em);
10        }
11        return em;
12    }
13 }
```

Listing 4 – Gestion du ThreadLocal pour l'EntityManager dans JPAUtil.java

6 Couche de Persistance : Implémentation des DAOs

6.1 Gestion Manuelle des Transactions

En l'absence de conteneur d'injection (comme Spring ou EJB), nous gérons manuellement le cycle de vie de chaque transaction de persistance via l'interface `EntityTransaction` du standard JPA. Les étapes respectent scrupuleusement le patron suivant :

1. Récupération de l'EntityManager associé au thread courant par la méthode utilitaire `JPAUtil.getEntityManager()`.
2. Démarrage de la transaction via `tx.begin()`.
3. Exécution des opérations d'écriture avec `persist` ou `merge`.
4. Validation de l'état persistant avec `tx.commit()`.
5. En cas d'exception, annulation via `tx.rollback()` et remontée de l'erreur.
6. Libération finale de l'EntityManager dans le bloc `finally`.

6.2 Le Patron Data Access Object (DAO)

Afin de découpler la couche d'accès aux données du reste de l'application, nous avons développé trois classes DAO principales : `AbsenceDAO`, `EtudiantDAO` et `ProfesseurDAO`. Elles encapsulent les requêtes JPQL (Java Persistence Query Language) et les opérations de recherche par clé primaire.

Pour illustrer l'implémentation de nos requêtes complexes, voici un extrait de la classe `AbsenceDAO.java` montrant la récupération des requêtes en attente de modération (justifications ou réclamations) :

```
1 public class AbsenceDAO {
2     public List<Absence> findPendingRequests() {
3         EntityManager em = JPAUtil.getEntityManager();
4         try {
5             return em.createQuery(
```

```

6         "SELECT a FROM Absence a WHERE a.
statusJustification = 'PENDING' OR a.statusReclamation = '
PENDING'",
7         Absence.class
8     ).getResultList();
9 } finally {
10     JPAUtil.closeEntityManager();
11 }
12 }
13 }

```

Listing 5 – Extrait des requêtes JPQL dans AbsenceDAO.java

7 Couche Contrôleur : La Servlet Front-Controller

La Servlet centralisée `AbsenceServlet.java` gère à elle seule le cycle de routage de la plateforme, vérifiant l'authentification et dispatchant les actions.

7.1 Vérification des Rôles et Session Gate

Afin d'éviter tout accès illégitime, la Servlet implémente un filtre directement dans les méthodes `doGet` et `doPost` :

- Extraction de la session sans création (`getSession(false)`).
- Si null, redirection vers le login.
- Si existante, comparaison de l'attribut `userRole`.

7.2 Structure Majeure du Servlet de Contrôle

Le servlet de contrôle centralise la logique de routage. Il analyse le paramètre `action` passé en paramètre d'URL (pattern Front-Controller) et aiguille la requête vers la méthode de traitement dédiée.

Afin d'illustrer la gestion des rôles à la connexion et le téléversement de fichiers multipart sans framework externe, voici les extraits clés de traitement :

```

1 private void processLogin(HttpServletRequest request,
    HttpServletResponse response)
2     throws ServletException, IOException {
3     String email = request.getParameter("email");
4     String password = request.getParameter("password");
5
6     // Authentification Admin
7     if ("admin@univ.fr".equals(email) && "admin".equals(password)
8 ) {
9         HttpSession session = request.getSession(true);
10        session.setAttribute("userRole", "ADMIN");
11        response.sendRedirect(request.getContextPath() + "/"
absences?action=list");
12        return;
13    }

```

```
13 // Traitement des roles PROFESSEUR et ETUDIANT via les DAOs  
correspondants...  
14 }
```

Listing 6 – Extrait de l'authentification unifiée dans AbsenceServlet.java

```
1 private void processStudentJustification(HttpServletRequest request,  
    HttpServletResponse response)  
2     throws ServletException, IOException {  
3     Long id = Long.parseLong(request.getParameter("id"));  
4     Part filePart = request.getPart("pieceJustificative");  
5     Absence abs = absenceDAO.findById(id);  
6  
7     if (abs != null && filePart != null && filePart.getSize() >  
0) {  
8         String fileName = filePart.getSubmittedFileName();  
9         String uniqueName = System.currentTimeMillis() + "_" +  
    fileName;  
10        String uploadPath = request.getServletContext().  
    getRealPath("/") + File.separator + "uploads";  
11  
12        filePart.write(uploadPath + File.separator + uniqueName);  
13        abs.setDemandeJustifFilePath("uploads/" + uniqueName);  
14        abs.setStatusJustification("PENDING");  
15        absenceDAO.save(abs);  
16    }  
17    response.sendRedirect(request.getContextPath() + "/absences?  
    action=studentDashboard");  
18 }
```

Listing 7 – Gestion de l'upload de justificatif dans AbsenceServlet.java

8 Gestion d'État : Sessions & Cookies

8.1 Statelessness HTTP et HttpSession

Le protocole HTTP est par nature sans état (stateless); chaque requête du client est traitée de façon totalement indépendante de la précédente. Pour maintenir la continuité de l'accès au tableau de bord, la plateforme s'appuie sur la classe standard `HttpSession`.

Lorsqu'un utilisateur s'authentifie, un identifiant de session unique (`JSESSIONID`) est stocké en mémoire serveur et partagé avec le navigateur sous forme de cookie temporaire. Ce cookie est transmis à chaque requête, permettant au serveur de charger les attributs associés à la session.

8.2 Persistance et Remplissage via Cookies

Pour améliorer le confort d'utilisation, un Cookie persistant est créé sur le client si la case "Se souvenir de moi" est cochée.

```
1 private void showLoginForm(HttpServletRequest request,  
    HttpServletResponse response)
```

```

2      throws ServletException, IOException {
3      Cookie[] cookies = request.getCookies();
4      if (cookies != null) {
5          for (Cookie c : cookies) {
6              if ("rememberedAdmin".equals(c.getName())) {
7                  request.setAttribute("rememberedAdmin", c.
getValue());
8                  break;
9              }
10         }
11     }
12     request.getRequestDispatcher("/WEB-INF/views/login.jsp").
forward(request, response);
13 }

```

Listing 8 – Lecture des cookies au chargement du login - AbsenceServlet.java

9 Téléversement de Fichiers & Exportations

9.1 Téléversement avec Multipart Config

Pour gérer les pièces jointes des étudiants de manière sécurisée, la Servlet est configurée avec l'annotation `@MultipartConfig` qui définit des contraintes de taille sur les téléversements. Les justificatifs sont sauvés dans un dossier physique `uploads/` au sein du répertoire de déploiement web.

9.2 Exportation CSV avec Signature BOM

L'exportation en format CSV est codée en écrivant directement dans le flux de sortie HTTP (`response.getOutputStream()`). Pour éviter que Microsoft Excel n'altère l'affichage des caractères accentués des noms d'étudiants (é, à, ç), nous écrivons la signature BOM UTF-8 (0xEF, 0xBB, 0xBF) avant d'initier l'écriture des chaînes de caractères.

10 Interface Utilisateur (UI) & CSS Glassmorphism

10.1 Philosophie du Design System

Pour proposer une esthétique digne des meilleures applications Web, nous avons défini un design Glassmorphic (effet verre dépoli) basé sur des couleurs sombres, des reflets lumineux et des flous d'arrière-plan.

```

1 :root {
2     --bg-primary: #0b0f19;
3     --accent-purple: #8b5cf6;
4     --accent-blue: #3b82f6;
5     --success: #10b981;
6     --danger: #ef4444;
7     --text-primary: #f8fafc;
8     --glass-bg: rgba(30, 41, 59, 0.45);
9     --glass-border: rgba(255, 255, 255, 0.08);

```

```
10 }
11 body {
12     background-color: var(--bg-primary);
13     color: var(--text-primary);
14     font-family: 'Inter', sans-serif;
15 }
16 .card {
17     background: var(--glass-bg);
18     backdrop-filter: blur(16px);
19     border: 1px solid var(--glass-border);
20     border-radius: 12px;
21 }
```

Listing 9 – Design Tokens - style.css

10.2 Filtrage client temps réel (Vanilla JS)

Le filtrage en temps réel dans les tableaux d'absences est effectué en JavaScript côté navigateur pour une interactivité instantanée.

```
1 function filterAbsences() {
2     var query = document.getElementById("searchInput").value.
        toLowerCase();
3     var filter = document.getElementById("statusFilter").value;
4     var rows = document.querySelectorAll("table tbody tr");
5
6     rows.forEach(function(row) {
7         var nom = row.cells[1].textContent.toLowerCase();
8         var cours = row.cells[2].textContent.toLowerCase();
9         var status = row.cells[4].textContent.toLowerCase();
10
11         var matchesQuery = nom.indexOf(query) > -1 || cours.
            indexOf(query) > -1;
12         var matchesStatus = filter === "ALL" ||
13             (filter === "JUSTIF" && status.indexOf("justifi") >
14              -1) ||
15             (filter === "NONJUSTIF" && status.indexOf("non
16              justifi") > -1);
17         row.style.display = (matchesQuery && matchesStatus) ? ""
18             : "none";
19     });
20 }
```

Listing 10 – Filtrage interactif

11 Structure Détaillée des Vues (JSPs)

La couche de présentation utilise des pages JSP (JavaServer Pages) associées à la bibliothèque de balises standard JSTL (JavaServer Pages Standard Tag Library). Cela

permet d'éviter l'écriture de code Java brut (scriptlets) au sein des fichiers HTML, préservant ainsi le modèle MVC.

11.1 Espace Étudiant - student-dashboard.jsp

Cette JSP présente les statistiques de l'étudiant connecté et liste ses absences à travers un tableau interactif. L'étudiant peut initier deux processus : le dépôt de justificatif et la réclamation.

La directive JSTL ci-dessous illustre le rendu conditionnel du bouton d'action selon le statut de l'absence :

```

1 <c:forEach var="abs" items="${absences}">
2   <tr>
3     <td>${abs.formattedDateSaisie}</td>
4     <td>${abs.seance.cours.nomCours}</td>
5     <td>${abs.estJustifiee ? 'Justifiee' : 'Non Justifiee'}</td>
6     <td>
7       <c:if test="${not abs.estJustifiee}">
8         <button onclick="openJustify(${abs.id})">
Justifier</button>
9       </c:if>
10    </td>
11  </tr>
12</c:forEach>

```

Listing 11 – Rendu conditionnel avec JSTL dans student-dashboard.jsp

11.2 Espace Enseignant - prof-dashboard.jsp

L'espace professeur implémente la feuille d'appel dynamique. L'enseignant choisit une séance, coche les étudiants absents et soumet le formulaire. La liste des étudiants est parcourue dynamiquement, et les identifiants cochés sont transmis en tableau de paramètres HTTP au servlet de contrôle :

```

1 <c:forEach var="st" items="${etudiants}">
2   <div class="student-item">
3     <span>${st.nomComplet}</span>
4     <input type="checkbox" name="absentStudents" value="${st.id}">
5   </div>
6</c:forEach>

```

Listing 12 – Boucle de sélection des étudiants pour l'appel

11.3 Espace Administration - admin-requests.jsp

Le portail de modération liste toutes les demandes reçues avec leurs motifs et les liens de téléchargement des justificatifs stockés en serveur. L'administrateur peut accepter ou rejeter chaque demande en un clic :

```
1 <c:forEach var="abs" items="${pendingAbsences}">
2   <div class="request-card">
3     <h3>${abs.etudiant.nomComple} - ${abs.seance.cours.
nomCours}</h3>
4     <p>Motif: ${abs.motifDemandeJustif}</p>
5     <a href="${pageContext.request.contextPath}/absences?
action=resolveJustification&id=${abs.id}&decision=accept">
Accepter</a>
6     <a href="${pageContext.request.contextPath}/absences?
action=resolveJustification&id=${abs.id}&decision=refuse">
Refuser</a>
7   </div>
8 </c:forEach>
```

Listing 13 – Actions de modération admin

12 Déploiement et Configuration

12.1 Configuration de déploiement (web.xml)

Le fichier ‘web.xml’ configure l’encodage des requêtes et le mapping d’URL racine ‘/absences’ pour diriger le trafic vers notre Servlet de contrôle unique.

12.2 Maven pom.xml

Les dépendances Maven intègrent les APIs de Servlets, JSPs, JSTL, Hibernate Core et le connecteur JDBC de PostgreSQL.

13 Scénarios d’Exécution et Recette de Tests

13.1 Test 1 : Authentification par rôle

- **Action** : Connexion avec admin@univ.fr / admin.
- **Résultat attendu** : Connexion réussie, accès au tableau de bord général de l’administration.

13.2 Test 2 : Appel interactif professeur

- **Action** : Connexion avec le compte enseignant. Clic sur une séance, cochage d’un étudiant comme absent, soumission.
- **Résultat attendu** : Les absences correspondantes sont générées automatiquement en base de données.

13.3 Test 3 : Dépôt de justificatif et validation

- **Action** : Connexion étudiante, sélection d’une absence, dépôt de motif et fichier. Connexion admin, examen de la demande dans le portail de modération, clic sur "Accepter".
- **Résultat attendu** : Le statut de l’absence passe à Justifiée dans le registre global.

14 Conclusion et Perspectives

L'architecture robuste de cette plateforme multi-rôles en Java EE 8 pur démontre la pertinence des standards natifs pour concevoir des applications modulaires et sécurisées. Par l'apprentissage des transactions JPA, de la sécurité par session et de l'interactivité native client, nous avons consolidé nos compétences fondamentales en ingénierie logicielle d'entreprise.